

Verification-Bounded Autonomy: A Two-System Architecture for Orchestrated Multi-Agent Work

Caleb Dean

Independent Developer & Researcher

github.com/loco306 · August 2026

ABSTRACT

We describe SUPERAGENT, a two-system architecture for autonomous multi-agent work built on a single premise: **the binding constraint on an autonomous agent is not its capability but the trustworthiness of its own success reports**. A language model that asserts it has finished is producing a token sequence, not evidence. Every architectural decision in both systems follows from treating that assertion as unreliable by default. We present two deployed systems sharing one doctrine but differing in verification substrate, and argue this split is not incidental — code and claims admit fundamentally different proof obligations.

1 Introduction

The dominant failure mode in autonomous agent systems is not incapacity. It is **confident completion of the wrong thing**, reported as success.

This failure is structurally invisible to the agent producing it. A model that has misread a requirement, silently skipped a step, or produced a plausible-but-wrong artifact has no internal signal distinguishing that state from success.

Let an agent produce artifact A and self-report r . The naive loop accepts A when $r = \text{success}$. The defect: A and r are generated by the same process under the same context. Their errors are **correlated**. Asking the agent whether it succeeded samples the same distribution twice.

SUPERAGENT is therefore best understood not as a system for *doing work* — models already do work — but as a system for **manufacturing trust in work that has already been done**.

2 Silent-Fail Signatures

Five recurring signatures, each observed in deployment:

- **Vacuous** — success reported; nothing produced.
- **No-op** — state that should have changed, didn't.
- **Stale** — output identical to a prior result when it should be fresh.

- **Skipped step** — work that completed suspiciously cheaply.
- **Verifier disagreement** — an independent check contradicts the producer.

The architectural response: **correctness is never owned by the agent that produced the work**. It is owned by a deterministic check, or by a fresh agent that did not build the thing and is instructed to refute it.

2.1 The Planted-Fault Standard

A verification regime is trustworthy only if it would catch a *deliberately* wrong result. If a planted fault would survive the check, the check is decorative. This is the calibration standard both systems are held to — run as a live drill, not as per-task overhead.

3 Design Principles

P1 — Done-conditions precede work. State a machine-checkable condition whose passing constitutes completion. If none can be written, the task is not yet specified. V1 states this as an ordering constraint: no code before its objective check exists.

P2 — Effort is derived, never fixed. Parallelism is a function of independent-piece count, capped at 16 concurrent subagents — a ceiling, never a default. A swarm costs roughly 15× main-loop tokens; every agent must earn its seat.

4 Two Systems, Two Proof Obligations

The central structural argument: code and claims are not the same kind of output. **Code is execution-checkable** — a test suite is a decision procedure, and correctness is local and mechanical. **A claim has no execution semantics**. No test runs a factual assertion; correctness is established only by corroboration across independently-derived sources. One architecture cannot optimize both.

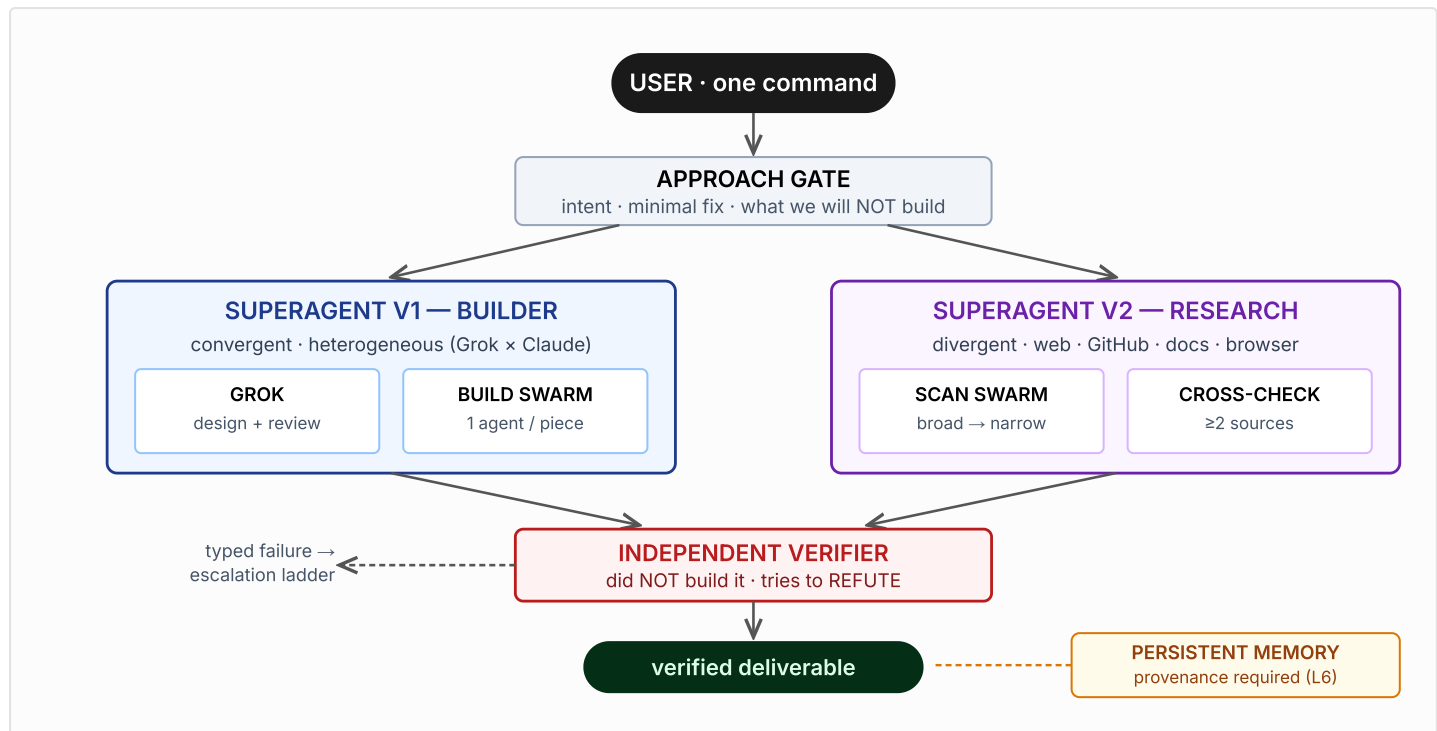


Figure 1. Both systems enter through a scope gate and exit through a verifier that did not produce the work. Failure is typed and routed to an escalation ladder in which each rung is a different *kind* of move — resume, retry, reformulate, decompose, switch — rather than a harder repetition of the last.

4.1 Comparative Structure

DIMENSION	V1 — BUILDER	V2 — RESEARCH
Output	Artifacts (code, tools)	Claims, findings
Topology	Convergent — pieces to one artifact	Divergent — parallel exploration
Verification	Execution + adversarial usability review	≥2 independent sources; single-source flagged
Decorrelation	Cross-vendor reviewer	Cross-source corroboration
Distinctive risk	Wrong-thing-built	Confident-but-unsourced claim

The systems are complements, not versions. V1 answers "*is it right?*" by running it. V2 answers "*is it true?*" by corroborating it. Neither question reduces to the other.